Residual Gated Dual Pooling for a Mini-PointNet: a controlled test

Haeyun Choi · DS 7400 Deep Learning, Homework 3 · code: `feature/hw3-gated-pooling`

1 Background and observed limitation

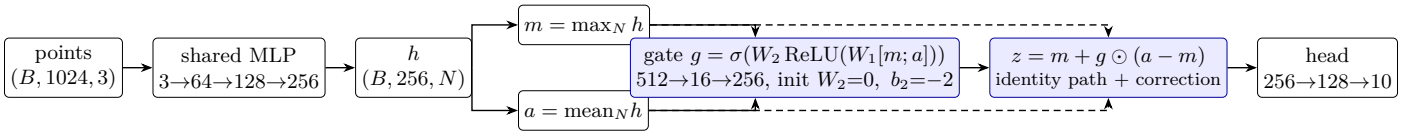
My HW2 classifier is a Mini-PointNet for ModelNet10 (10 furniture classes; 3,193 train / 798 validation / 908 test objects). Each CAD mesh becomes **1,024 points sampled on its surface** (triangles chosen $\propto$ area); a point is only (x, y, z), so a batch is a tensor of shape $(B, 1024, 3)$. Points are centred and scaled into the unit sphere ($\hat{p}_i = (p_i - \mu)/\max_j \|p_j - \mu\|$), which removes translation *and absolute size*; training adds an independent per-axis scale $\mathcal{U}(0.8, 1.25)$, a shift and clipped jitter. A shared per-point MLP (3→64→128→256, Conv1d k=1, BatchNorm, ReLU) produces $h_i \in \mathbb{R}^{256}$, a **global max pool** $m_c = \max_i h_{i,c}$ makes the result order-invariant, and a head 256→128→10 classifies (76,362 parameters; 92.77±0.60% test accuracy over 3 seeds). Its errors concentrate on look-alike pairs: in seed 0, **12 tables→desk, 4 desks→table, 11 dressers→night stand and 11 night stands→dresser** — 38 of 58 errors. These pairs differ mostly in proportion and extent. Max pooling keeps only the most extreme point per channel: it ignores *how many* points show a feature (prevalence), how surface features are distributed, and $\partial m_c/\partial h_{i,c} = \mathbf{1}[i = \arg\max]$ routes gradient to one point per channel.

2 Hypothesis and proposed design (fixed before any HW3 run)

Hypothesis. A learned, channel-wise blend of max and mean statistics improves the proportion-defined pairs; this is a testable guess, not a known cause. Two course principles shape the design: **ResNet** [1] — keep a mapping that works as the identity path and learn a *correction* around it; **SENet** [2] — squeeze global descriptors and learn a data-dependent *channel-wise* sigmoid gate. **Residual Gated Dual Pooling** (E2) keeps the max-pooled vector and adds a gated move toward the mean:

$$m = \max_N h, \quad a = \text{mean}_N\, h, \quad g = \sigma\big(W_2\,\text{ReLU}(W_1[m; a] + b_1) + b_2\big), \quad z = m + g \odot (a - m) = (1 - g) \odot m + g \odot a.$$

W_1: 512→16, W_2: 16→256 (SENet's reduction ratio 16), +12,560 parameters. **Baseline-preserving initialisation**: W_2=0, b_2=−2, so training starts at $g = \sigma(-2) \approx 0.119$ for every channel and object — near max pooling, but outside sigmoid saturation. The head is unchanged.



3 Controlled experiment

Four conditions, seeds 0, 1, 2 (12 runs): **E0** the HW2 model rerun (max pool); **E1** fixed dual pool $z = 0.5m + 0.5a$ (no new parameters: does mean information help at all?); **E2** the proposed module (does *learned* selection help?); **E3** max pool with width h=69, 88,537 parameters (is an E2 gain just its +16% parameters?). Held fixed: the cached 1,024 points per object and splits, augmentation, 60 epochs, batch 32, Adam (lr 10^{-3}, weight decay 10^{-4}), cosine schedule, dropout 0.3, checkpoint by validation accuracy, one test evaluation, package versions and CPU model (Xeon Gold 6248, 8 threads). Within a seed every condition sees the same batches and augmentation, and E0/E1/E2 start from identical shared weights and dropout stream (the gate is built in a forked RNG state) — only the pooling differs. Decision rule written in advance: *supported* only if E2 beats both E0 and E3 by ≥0.5 pp (HW2 seed SD ≈0.6 pp; one test object = 0.11 pp) and the critical-pair errors fall.

4 Results (post-experiment observations)

Verdict: the hypothesis is not supported. Implementation and training completed normally; E0 reproduces the stored HW2 runs *exactly* (93.61 / 92.29 / 92.40%, same selected epochs), so differences between conditions reflect the change or seed variability, not environment drift.

Model	Params	Val acc.	**Test acc.**	table →desk	desk →table	dresser →night st.	night st. →dresser	Test acc. 256 points
E0 max pool (HW2)	76,362	97.70	**92.77** ± 0.60	16.0	4.3	11.0	11.3	89.61 ± 0.54
E1 fixed dual	76,362	97.66	**92.66** ± 0.26	11.3	4.7	11.7	12.3	89.83 ± 0.05
E2 gated dual (ours)	88,922	97.49	**92.77** ± 0.32	16.7	4.3	8.3	15.3	**90.75** ± 0.18
E3 max pool, h=69	88,537	97.41	**91.81** ± 0.34	19.7	4.3	15.7	12.0	89.24 ± 0.81

Table 1: mean (± SD) over 3 seeds, %; error columns are test objects (mean over seeds); last column: exploratory evaluation with 256 points per object.

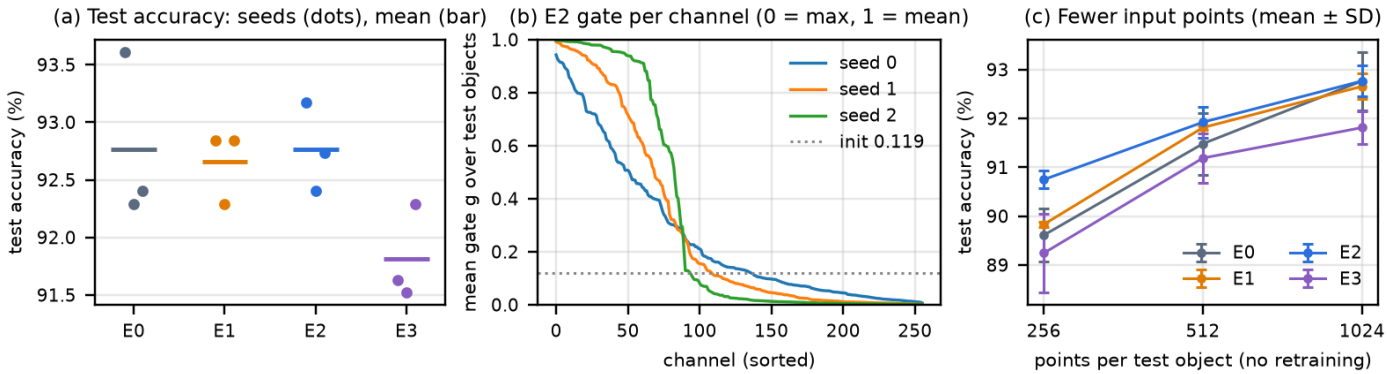


Figure 1: (a) test accuracy of every run; (b) E2's mean gate per channel, sorted (0 = max, 1 = mean pooling); (c) the same checkpoints evaluated on fixed 512/256-point subsets (exploratory).

Confirmatory (decision rules fixed in advance):

- **No clean-accuracy gain.** Paired by seed, E2−E0 = −0.44, +0.11, +0.33 pp (mean **+0.00**); exact two-sided McNemar test on the same 908 test objects, one test per seed (seeds never pooled): $p = 0.57, 1.00, 0.61$. E1−E0 = −0.11 pp; E2−E1 = +0.11 pp. E3 performed *worse* (−0.95 pp, 3/3 seeds), so added parameters alone did not help.
- **Errors redistributed, not reduced.** E2 lowered dresser→night stand (11.0→8.3) but raised night stand→dresser (11.3→15.3); E1 lowered table→desk (16.0→11.3) but no other direction. Four-direction total: E0 42.7, E1 40.0, E2 44.7 — no consistent reduction across seeds.
- **The learned gate did not behave as hypothesized.** Instead of an object-adaptive mixture, optimization produced a nearly static, bimodal channel split (b): from g=0.119, 52–83 of 256 channels moved toward mean pooling (mean $g > 0.5$) and 110–160 toward pure max (mean $g < 0.1$); weight decay, which pulls g toward 0.5, cannot produce this split. A channel's gate barely varies across objects (SD 0.006–0.017 vs. 0.27–0.42 across channels) and class means differ by <0.01. What failed is the adaptive-mixture hypothesis, not the implementation.

Exploratory (not a confirmatory test): with one fixed 256-point subset per object, E2 scores 90.75% vs. 89.61% for E0, higher in each seed (c); a max over fewer points can only shrink, a mean does not. This suggests that the proposed pooling *may* be more robust when fewer points are available.

5 Failure analysis and next experiment

Why pooling may not have helped these pairs. A post-hoc probe (not pre-registered; no network, no retraining) asks how well a single global cue separates each pair: the best threshold on one feature, chosen on the training pool, scored on the test split as balanced accuracy (50% = chance).

Pair	Cue	Test bal. acc.	Affected by
dresser vs. night stand	absolute height (raw mesh units)	**85.5%**	normalisation (never in the input)
dresser vs. night stand	depth/width ratio	81.4% → **73.3%**	per-axis scale augmentation (training)
table vs. desk	any size or bounding-box ratio	42–56%	— (not a global cue)

Absolute size never reaches the network, so no aggregator can use it; the dresser/night-stand confusion is consistent with this, since absolute height alone is almost as good as the network's own within-pair accuracy (≈87%). The depth/width cue reaches the network at test time but is perturbed in training, which may teach the network to discount it. For table/desk no global statistic helps, which suggests local structure (drawers, panels) — what max pooling already targets — or label ambiguity. These are diagnostics consistent with, not proof of, the alternative explanations.

Alternative explanations kept open. Seed variability is large: paired runs that differ only in pooling still flip 15–39 test objects, so three seeds resolve about 1 pp, not 0.1 pp, and E3 shows that changing initialisation and width alone moves the score by ∼1 pp (HW2's 93.6% seed 0 was a favourable draw). The gate may also need an input that truly varies per object (e.g. the normalisation radius) to become data-dependent.

Next controlled experiment. *E4*: E0 plus one scalar, $\log r$ (the normalisation radius, i.e. absolute size), concatenated to the pooled feature (+128 parameters), seeds 0–2, everything else fixed; primary outcome: dresser↔night-stand errors. Then *E5*: isotropic scale augmentation (one factor for all axes) to test the augmentation explanation alone. Caveat: ModelNet sizes are per-model CAD units, and a size-aware model depends on the units of its inputs.

Lesson. A well-motivated module can train correctly and still leave the target errors unchanged. The controls (E1, E3), the gate statistics and a cheap probe suggest that the limitation may lie in the input pipeline rather than in the aggregation, and pointed to a possible side effect (robustness to sparse inputs) that clean accuracy hides.

References. [1] K. He, X. Zhang, S. Ren, J. Sun, Deep Residual Learning for Image Recognition, CVPR 2016. [2] J. Hu, L. Shen, G. Sun, Squeeze-and-Excitation Networks, CVPR 2018. [3] Z. Wu et al., 3D ShapeNets (ModelNet), CVPR 2015. [4] C. R. Qi et al., PointNet, CVPR 2017. Code, all 12 runs and logs: `README.md`, `docs/hw3-design.md` (committed before training), `docs/hw3-experiments.md`, `results/hw3/`.